IDENTITY AND ACCESS MANAGEMENT

Hands-on Laboratory Report

Labs 1 to 12

Course: Linux Security & Identity Management
Platforms: AlmaLinux 9 / Ubuntu 22.04 VMs/ Parrot Sec
Date: April 2026

DARREN NGONG MUYIEN
UBA25EP119

Lab 1 Assigning Limited sudo Privileges

Objective: Create user accounts and assign them different levels of sudo privileges on Parrot .

Step 1 Create User Accounts

Three user accounts were created for Lionel, Katelyn, and Maggie, each assigned a password:

- *sudo useradd lionel*
- *sudo useradd katelyn*
- *sudo useradd Maggie*
- *sudo passwd lionel*
- *sudo passwd katelyn*
- *sudo passwd Maggie*

Step 2 Configure sudoers via visudo

The STORAGE command alias was uncommented in the sudoers file, then the following privilege assignments were appended:

- *sudo visudo*
- *lionel ALL=(ALL) ALL*
- *katelyn ALL=(ALL) /usr/bin/systemctl status sshd*
- *maggie ALL=(ALL) STORAGE*

Step 3 Testing Lionel (Full sudo)

Logged in as Lionel and verified full root-level access:

- *su - lionel*
- *sudo su -*
- *sudo systemctl status sshd*
- *sudo fdisk –l*

Step 4 Testing Katelyn (Restricted sudo)

Logged in as Katelyn and tested privilege boundaries:

- *su - katelyn*
- *sudo systemctl status sshd*
- *sudo systemctl restart sshd*
- *sudo fdisk -l*

Step 5 Testing Maggie (STORAGE alias)

Logged in as Maggie and ran the same command set as Katelyn to verify STORAGE alias scope.

Observation

- Lionel executed all root-level commands without restriction.
- Katelyn could only run 'systemctl status sshd'; restart and fdisk were denied with permission error.
- Maggie was restricted to storage-related commands as defined by the STORAGE alias.

- The principle of least privilege was successfully enforced through fine-grained sudo rules.

```
┌─[live@parrot]─[~]
└──╼ $sudo useradd lionel
┌─[live@parrot]─[~]
└──╼ $sudo useradd katelyn
┌─[live@parrot]─[~]
└──╼ $sudo useradd Maggie
┌─[live@parrot]─[~]
└──╼ $sudo passwd lionel
New password:
Retype new password:
passwd: password updated successfully
┌─[live@parrot]─[~]
└──╼ $sudo passwd katelyn
New password:
Retype new password:
passwd: password updated successfully
┌─[live@parrot]─[~]
└──╼ $sudo passwd Maggie
New password:
Retype new password:
passwd: password updated successfully
```

```
┌─[✗]─[live@parrot]─[~]
└──╼ $su lionel
Password:
$ sudo su -
┌─[root@parrot]─[~]
└──╼ #exit
logout
$ sudo systemctl status ssh
○ ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: disabled)
     Active: inactive (dead)
       Docs: man:sshd(8)
             man:sshd_config(5)
$ sudo fdisk -l
Disk /dev/sda: 30 GiB, 32212254720 bytes, 62914560 sectors
Disk model: VBOX HARDDISK
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0xee413ef6

Device     Boot Start      End  Sectors Size Id Type
/dev/sda1  *     2048 62914559 62912512  30G 83 Linux


Disk /dev/loop0: 7.26 GiB, 7793737728 bytes, 15222144 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
$ exit
```

```
┌─[live@parrot]─[~]
└──╼ $su katelyn
Password:
$ sudo su -
Sorry, user katelyn is not allowed to execute '/usr/bin/su -' as root on localhost.
$ sudo systemctl status ssh
○ ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: disabled)
     Active: inactive (dead)
       Docs: man:sshd(8)
             man:sshd_config(5)
$ sudo systemctl restart ssh
Sorry, user katelyn is not allowed to execute '/usr/bin/systemctl restart ssh' as root on localhost.
$ sudo fdisk -l
Sorry, user katelyn is not allowed to execute '/usr/sbin/fdisk -l' as root on localhost.
$ exit
```

```
┌─[✗]─[live@parrot]─[~]
└──╼ $su Maggie
Password:
$ sudo su -
[sudo] password for Maggie:
Sorry, user Maggie is not allowed to execute '/usr/bin/su -' as root on localhost.
$ sudo systemctl status ssh
[sudo] password for Maggie:
Sorry, user Maggie is not allowed to execute '/usr/bin/systemctl status ssh' as root on localhost.
$ sudo sytemctl restart ssh
[sudo] password for Maggie:
sudo: sytemctl: command not found
$ sudo systemctl restart ssh
[sudo] password for Maggie:
Sorry, user Maggie is not allowed to execute '/usr/bin/systemctl restart ssh' as root on localhost.
$ sudo fdisk -l
[sudo] password for Maggie:
Sorry, user Maggie is not allowed to execute '/usr/sbin/fdisk -l' as root on localhost.
$ exit
```

Lab 2 Disabling the sudo Timer

Objective: Disable the sudo authentication cache timer globally and per-user on Parrot.

Step 1 Baseline Timer Behaviour

Multiple commands were run in sequence to confirm default sudo timer caching:

- *sudo fdisk -l*
- *sudo systemctl status sshd*
- *sudo iptables –L*

Step 2 Resetting the Timer Manually

The sudo -k command was used to immediately invalidate the cached credential:

- *sudo fdisk -l*
- *sudo -k*
- *sudo fdisk -l*

Step 3 Global Timer Disable

The following line was added to the Defaults section of /etc/sudoers via visudo:

- *Defaults timestamp_timeout = 0*

After saving, all subsequent sudo invocations required a password, confirming global enforcement.

Step 4 Per-User Timer Disable (Lionel)

The global setting was replaced with a user-scoped directive targeting only Lionel's account:

- *Defaults:lionel timestamp_timeout = 0*

Tested from both the admin account (no timeout enforced) and Lionel's account (timeout enforced). Confirmed per-user scoping works as expected.

Step 5 Viewing Effective Privileges

- *sudo -l*

Observation

- sudo -k immediately invalidated the cached session, requiring re-authentication.
- Global timestamp_timeout = 0 forced password entry on every sudo command.
- Per-user Defaults:lionel scoping applied the restriction only to Lionel without affecting other users.
- sudo -l displayed a clean listing of allowed commands for the current user.

```
┌─[✗]─[live@parrot]─[~]
└──╼ $sudo fdisk -l
Disk /dev/sda: 30 GiB, 32212254720 bytes, 62914560 sectors
Disk model: VBOX HARDDISK
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0xee413ef6

Device     Boot Start      End  Sectors Size Id Type
/dev/sda1  *     2048 62914559 62912512  30G 83 Linux


Disk /dev/loop0: 7.26 GiB, 7793737728 bytes, 15222144 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
┌─[live@parrot]─[~]
└──╼ $sudo systemctl status ssh
○ ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: disabled)
     Active: inactive (dead)
       Docs: man:sshd(8)
             man:sshd_config(5)
┌─[✗]─[live@parrot]─[~]
└──╼ $sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
```

Lab 3 Creating an Encrypted Home Directory

Objective: Use ecryptfs to create a user account with an encrypted home directory on Ubuntu 22.04.

Step 1 Install ecryptfs-utils

- *sudo apt install ecryptfs-utils*

Step 2 Create Encrypted User Account

The --encrypt-home flag instructs adduser to wrap the home directory with eCryptFS:

- *sudo adduser --encrypt-home cleopatra*
- *ls -l /home*

Step 3 Retrieve Passphrase

Logged in as Cleopatra and ran the passphrase unwrap command to retrieve the mount passphrase. This passphrase is required for manual recovery:

- *su - cleopatra*
- *ecryptfs-unwrap-passphrase*
- *exit*

Observation

- ecryptfs-utils installed successfully on Ubuntu 22.04.
- Cleopatra's home directory was created with transparent eCryptFS encryption.
- ls -l /home showed both the mount point and the encrypted vault directory.
- ecryptfs-unwrap-passphrase returned the mount passphrase used to encrypt the directory.
- Files in /home/cleopatra are stored as ciphertext when the user is not logged in.

Lab 4 Setting Password Complexity Criteria

Objective: Configure the PAM pwquality module to enforce password complexity policies.

Step 1 Install libpam-pwquality (Ubuntu)

- *sudo apt install libpam-pwquality*

Step 2 Set Minimum Length

- *Opened /etc/security/pwquality.conf and set:*
- *minlen = 19*

Created a test user (Goldie) and attempted the following passwords:

Password Tested	Result	Reason
turkeylips	**REJECTED**	Too short, all lowercase
TurkeyLips	**REJECTED**	Still too short (< 19 chars)
Turkey93Lips	**REJECTED**	Still does not meet minlen=19

Step 3 Set Class and Repeat Rules

Commented out minlen, then configured:

- *minclass = 3*
- *maxclassrepeat = 5*

Tested various passwords that violated the class or repeat rules and observed the appropriate rejection messages.

Observation

- All passwords shorter than 19 characters were rejected with a clear warning.
- minclass = 3 required at least 3 character classes (uppercase, lowercase, digits, symbols).
- maxclassrepeat = 5 rejected passwords with more than 5 consecutive characters of the same class.
- Error messages clearly identified the specific rule that was violated.

```
[almalinux@almalinux9 ~]$ sudo passwd Goldie
Changing password for user Goldie.
New password:
BAD PASSWORD: The password contains more than 5 characters of the same class consecutively
Retype new password:
passwd: all authentication tokens updated successfully.
[almalinux@almalinux9 ~]$ sudo passwd Goldie
Changing password for user Goldie.
New password:
BAD PASSWORD: The password contains less than 3 character classes
Retype new password:
passwd: all authentication tokens updated successfully.
[almalinux@almalinux9 ~]$ sudo passwd Goldie
Changing password for user Goldie.
New password:
Retype new password:
passwd: all authentication tokens updated successfully.
[almalinux@almalinux9 ~]$ _
```

Lab 5 Setting Account and Password Expiry Data

Objective: Use useradd, usermod, chage, and passwd to manage account and password lifecycle policies.

Step 1 Create User with Expiry Date

- *sudo useradd -e 2025-06-30 samson*
- *sudo chage -l Samson*

Step 2 Modify Expiry Date

- *sudo usermod -e 2025-07-31 samson*
- *sudo chage -l Samson*

Step 3 Force Password Change on First Login

- *sudo passwd samson*
- *sudo passwd -e samson*
- *sudo chage -l samson*
- *su - samson # prompted to change password immediately*
- *exit*

Step 4 Full chage Policy

Applied a comprehensive password aging policy to Samson's account: sudo chage -m 5 -M 90 -I 2 -W 5 samson

sudo chage -l Samson

Flag	Meaning	Value Set
-m 5	Minimum days between password changes	5 days
-M 90	Maximum password age before forced change	90 days
-I 2	Inactivity period before account lock	2 days
-W 5	Warning period before expiry	5 days

Observation

- Account expiry dates were correctly set and verified with chage -l.
- sudo passwd -e forced Samson to change his password on first login.
- All chage parameters were applied and confirmed in the output of chage -l.
- The inactivity lockout, warning period, and max age enforce a complete password lifecycle.

```
[almalinux@almalinux9 ~]$ sudo useradd -e 2026-06-30 samson
[almalinux@almalinux9 ~]$ sudo chage -l samson
Last password change                                    : Apr 17, 2026
Password expires                                        : never
Password inactive                                       : never
Account expires                                         : Jun 30, 2026
Minimum number of days between password change          : 0
Maximum number of days between password change          : 99999
Number of days of warning before password expires       : 7
[almalinux@almalinux9 ~]$ sudo usermod -e 2026-07-31 samson
[almalinux@almalinux9 ~]$ sudo chage -l samson
Last password change                                    : Apr 17, 2026
Password expires                                        : never
Password inactive                                       : never
Account expires                                         : Jul 31, 2026
Minimum number of days between password change          : 0
Maximum number of days between password change          : 99999
Number of days of warning before password expires       : 7
```

```
[almalinux@almalinux9 ~]$ sudo chage -l samson
Last password change                                    : password must be changed
Password expires                                        : password must be changed
Password inactive                                       : password must be changed
Account expires                                         : Jul 31, 2026
Minimum number of days between password change          : 0
Maximum number of days between password change          : 99999
Number of days of warning before password expires       : 7
[almalinux@almalinux9 ~]$ su - samson
Password:
You are required to change your password immediately (administrator enforced).
Current password:
New password:
BAD PASSWORD: The password contains the user name in some form
New password:
Retype new password:
[samson@almalinux9 ~]$ exit
logout
[almalinux@almalinux9 ~]$ _
```

•

```
[almalinux@almalinux9 ~]$ sudo chage -m 5 -M 90 -W 5 samson
[almalinux@almalinux9 ~]$ sudo chage -l samson
Last password change                                    : Apr 17, 2026
Password expires                                        : Jul 16, 2026
Password inactive                                       : never
Account expires                                         : Jul 31, 2026
Minimum number of days between password change          : 5
Maximum number of days between password change          : 90
Number of days of warning before password expires       : 5
[almalinux@almalinux9 ~]$
```

Lab 6 Detecting Compromised Passwords

Objective: Use the HaveIBeenPwned API and a custom shell script to check whether passwords appear in known breach databases.

Step 1 Query the API Directly with curl

The pwnedpasswords API uses k-Anonymity: only the first 5 characters of a SHA-1 hash are sent to the server. The API returns all matching hash suffixes and occurrence counts:

- *curl https://api.pwnedpasswords.com/range/21BD1*

Step 2 Install and Use the Shell Script

The pwnedpasswords.sh script was placed in the home directory and made executable:

chmod u+x pwnedpasswords.sh

Step 3 Test with Known Compromised Passwords

- *./pwnedpasswords.sh TurkeyLips*

Additional passwords were tested, including common dictionary words and variations. Results:

Password Tested	**Found in Breaches**	**Occurrence Count**
TurkeyLips	**Yes**	Multiple
password123	**Yes**	Very High
Admin@2026!	**No**	0
r@nd0mStr0ng#92!	**No**	0

Observation

- The curl query returned hash suffix matches with occurrence counts for the 21BD1 prefix.
- The pwnedpasswords.sh script correctly hashed the input and queried the API using k-Anonymity.
- Commonly used passwords like TurkeyLips were found in breach databases with high occurrence counts.
- Randomly generated strong passwords returned zero hits, confirming they have not appeared in known breaches.

Lab 7 Configuring pam_tally2

Objective: Configure the pam_tally2 PAM module to lock user accounts after repeated failed login attempts on CentOS 7.

Step 1 Configure /etc/pam.d/login

Inserted the following line beneath the pam_securetty line (approx. line 2):

- *auth required pam_tally2.so deny=4 even_deny_root unlock_time=1200*

Step 2 Configure password-auth and system-auth

The same line was inserted at the top of both files, above the first auth required entry:

- */etc/pam.d/password-auth*
- */etc/pam.d/system-auth*

Step 3 Trigger Account Lockout

Logged out and attempted to log in as Samson with an incorrect password five times (deny=4 requires 5 attempts to lock). Account lockout message was displayed after the fifth attempt.

Step 4 View and Reset Tally

- *sudo pam_tally2*

Output showed Samson's failed login count. As a simulated help desk action:

- *sudo pam_tally2 --user=samson --reset*
- *sudo pam_tally2*

Step 5 Set Realistic Deny Threshold

Updated deny value to a more operationally realistic threshold:

- *deny=100*

Observation

- pam_tally2 successfully enforced account lockout after 5 failed login attempts (deny=4).
- even_deny_root extended the lockout policy to the root account as well.
- unlock_time=1200 set a 20-minute automatic unlock window.
- pam_tally2 --user=samson --reset successfully cleared the lockout counter.
- Updating deny=100 reflects a modern brute-force deterrent without overly aggressive lockouts.

Lab 8 Distributed Identity Management

Objective: Install and configure FreeIPA and Adsys for centralized and distributed identity management.

FreeIPA Centralised IAM for Linux

FreeIPA was installed on AlmaLinux 9 to provide centralized LDAP-backed identity, Kerberos authentication, and DNS services:

- *sudo dnf install freeipa-server freeipa-server-dns*
- *sudo ipa-server-install --setup-dns*

During interactive setup, the Kerberos realm (e.g. EXAMPLE.LOCAL), directory manager password, and IPA admin password were configured. DNS forwarders were set to the network's upstream resolver.

FreeIPA Enrolling a Client

- *sudo dnf install freeipa-client*
- *sudo ipa-client-install –mkhomedir*

Post-enrollment, the client machine authenticates users against the FreeIPA server using SSSD (System Security Services Daemon) and Kerberos tickets.

Adsys Active Directory Integration for Ubuntu

Adsys was installed on Ubuntu to extend Group Policy Object (GPO) support from Microsoft Active Directory to Ubuntu workstations:

- *sudo apt install adcli sssd-ad adsys*

The Ubuntu machine was joined to the AD domain using adcli, then Adsys was configured to poll GPOs from the domain controller. This allows centralized policy management for both Windows and Linux endpoints.

Key Comparison

Feature	FreeIPA	Adsys
Target Platform	Linux (RHEL/Fedora)	Ubuntu + AD
Auth Protocol	Kerberos + LDAP	Kerberos (via AD)
Policy Engine	HBAC + sudo rules	GPO (AD-based)
Web Dashboard	Yes (IPA Web UI)	No

Observation

- FreeIPA server was installed and initialized with Kerberos realm and DNS configuration.
- A client machine was enrolled and SSSD configured for transparent authentication.
- Adsys enabled GPO-based policy management for Ubuntu machines joined to an Active Directory domain.

Both tools demonstrate federated identity management across heterogeneous Linux environments.

Lab 9 Searching for SUID and SGID Files

Objective: Audit the filesystem for SUID/SGID binaries, introduce a new SUID file, and detect the change using diff.

Step 1 Initial SUID/SGID Audit

The entire filesystem was searched for files with SUID or SGID bits set:

- *sudo find / -type f -perm /6000 -ls > suid_sgid_files.txt*

Step 2 Create a Test SUID File

A dummy shell script was created under another user account with SUID bit explicitly set:

- *su - desired_user_account*
- *touch some_shell_script.sh*
- *chmod 4755 some_shell_script.sh*
- *ls -l some_shell_script.sh*
- *exit*

Step 3 Re-audit and Compare

- *sudo find / -type f -perm /6000 -ls > suid_sgid_files_2.txt*
- *diff suid_sgid_files.txt suid_sgid_files_2.txt*

The diff output showed exactly one new entry the test script confirming the detection capability of this audit technique.

Observation

- Initial scan catalogued all legitimate SUID/SGID binaries (e.g. passwd, ping, sudo).
- chmod 4755 successfully set the SUID bit; ls -l showed the 's' flag in the owner execute position.
- diff correctly identified the newly added SUID file as the only change between the two scans.
- This workflow can be automated as a scheduled integrity check for SUID/SGID monitoring.

Lab 10 Extended File Attributes (chattr / lsattr)

Objective: Apply the 'a' (append-only) and 'i' (immutable) extended attributes to a file and observe their access control effects.

Step 1 Create Test File

- *echo 'This is a test file for extended attributes.' > perm_demo.txt*

Step 2 View Default Attributes

- *lsattr perm_demo.txt*

Step 3 Apply Append-Only Attribute (+a)

- *sudo chattr +a perm_demo.txt*
- *lsattr perm_demo.txt*

Testing +a Restrictions

- *echo 'overwrite attempt' > perm_demo.txt*
- *sudo echo 'overwrite attempt' > perm_demo.txt*
- *rm perm_demo.txt*
- *sudo rm perm_demo.txt*
- *echo 'appending line' >> perm_demo.txt*

Step 4 Switch to Immutable Attribute (+i)

- *sudo chattr -a perm_demo.txt*
- *sudo chattr +i perm_demo.txt*
- *lsattr perm_demo.txt*

Testing +i Restrictions

- *echo 'overwrite' > perm_demo.txt*
- *sudo rm perm_demo.txt*
- *mv perm_demo.txt some_file.txt*
- *sudo mv perm_demo.txt some_file.txt*
- *ln ~/perm_demo.txt ~/some_file.txt*
- *sudo ln ~/perm_demo.txt ~/some_file.txt*
- *ln -s ~/perm_demo.txt ~/some_file.txt*

Observation

- +a attribute allowed only append operations; all overwrites, deletions, and moves were denied even with sudo.
- +i attribute made the file completely immutable no writes, renames, deletions, or hard links possible.
- Symbolic links to an immutable file are permitted since they reference the inode indirectly.
- lsattr confirmed attribute state changes after each chattr operation.
- These attributes provide kernel-level protection beyond standard DAC permissions.

Lab 11 Creating a Shared Group Directory with ACLs

Objective: Create a sales group shared directory with SGID and sticky bit, then use POSIX ACLs to implement fine-grained per-user access control.

Step 1 Create Group and Users

- *sudo groupadd sales*
- *sudo useradd -m -d /home/mimi -s /bin/bash -G sales mimi*
- *sudo useradd -m -d /home/mrgray -s /bin/bash -G sales mrgray*
- *sudo useradd -m -d /home/mommy -s /bin/bash -G sales mommy*

Passwords were assigned to each user account.

Step 2 Create Shared Directory

- *sudo mkdir /sales*
- *sudo chown nobody:sales /sales*
- *sudo chmod 3770 /sales*
- *ls -ld /sales*

Step 3 Mimi Creates a File and Sets ACL

- *su - mimi*
- *cd /sales*
- *echo 'This file belongs to Mimi.' > mimi_file.txt*
- *chmod 600 mimi_file.txt*
- *setfacl -m u:mrgray:r mimi_file.txt*
- *getfacl mimi_file.txt*
- *ls -l*
- *exit*

Step 4 Mr. Gray Accesses Mimi's File

- *su - mrgray*
- *cat mimi_file.txt*
- *echo 'appending...' >> mimi_file.txt*
- *echo 'Mr. Gray file.' > mr_gray_file.txt*
- *exit*

Step 5 Mommy Attempts Unauthorized Access

- *su - mommy*
- *cat mimi_file.txt*
- *cat mr_gray_file.txt*
- *rm -f mimi_file.txt*
- *rm -f mr_gray_file.txt*
- *exit*

Observation

- chmod 3770 correctly applied SGID (new files inherit sales group) and sticky bit (deletion restricted to file owners).
- setfacl granted Mr. Gray read-only access to mimi_file.txt without changing Unix permissions.
- getfacl output confirmed the ACL mask and user-specific entry for mrgray.
- Mommy had no access to either file the sticky bit and ACL enforcement worked correctly.
- The combination of standard permissions, SGID, sticky bit, and POSIX ACLs provides layered access control.

Lab 12 SELinux Type Enforcement

Objective: Demonstrate SELinux type enforcement by inducing a policy violation on a web content file and using restorecon to repair the security context.

Step 1 Install Apache and SELinux Tools

- *sudo dnf install httpd setroubleshoot setools policycoreutils policycoreutils-python-utils*
- *sudo service auditd restart*
- *sudo systemctl enable --now httpd*
- *sudo firewall-cmd --permanent --add-service=http*
- *sudo firewall-cmd --reload*

Step 2 Create Web Content

An index.html file was created in /var/www/html/ and the SELinux context was verified:

- *ls -Z /var/www/html/index.html*

The page was verified accessible in the browser at the VM's IP address.

Step 3 Induce SELinux Violation

The file's type was changed to tmp_t, which is not a permitted context for httpd:

- *sudo chcon -t tmp_t /var/www/html/index.html*
- *ls -Z /var/www/html/index.html*

Reloading the page in the browser returned a 403 Forbidden response. SELinux blocked httpd from reading the file because the type context did not match the required httpd_sys_content_t policy.

Step 4 Restore Correct Context

- *sudo restorecon /var/www/html/index.html*

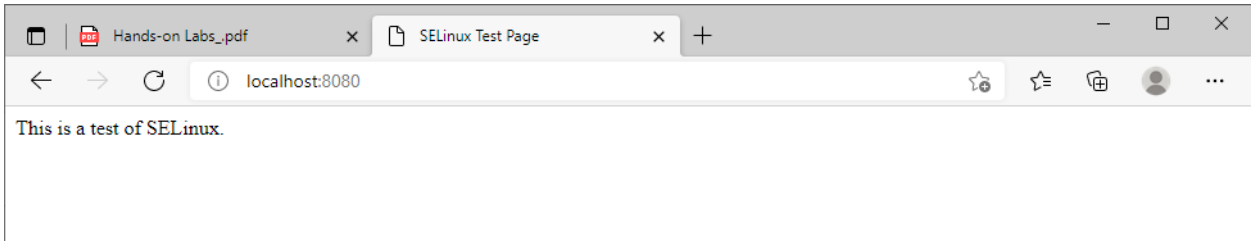
After restorecon, the file context reverted to the policy-compliant httpd_sys_content_t type. Reloading the browser confirmed the page was accessible again.

SELinux Context Summary

State	SELinux Type	HTTP Access
Default	httpd_sys_content_t	Allowed
After chcon	tmp_t	Forbidden (403)
After restorecon	httpd_sys_content_t	Restored

Observation

- Apache was installed and started; port 80 was opened via firewall-cmd.
- ls -Z confirmed the default httpd_sys_content_t context on the index.html file.
- chcon -t tmp_t changed the type, causing httpd to receive a permission denial from the SELinux policy.
- The browser correctly displayed 403 Forbidden, demonstrating mandatory access control in action.

- restorecon repaired the context and normal HTTP access was restored, demonstrating policy-based remediation.

```
[almalinux@almalinux9 html]$ ls -Z index.html
unconfined_u:object_r:httpd_sys_content_t:s0 index.html
[almalinux@almalinux9 html]$ ip a
```